Coding & Solution

Wandr AI – AI Travel Itinerary Planner

1. Backend Key Implementation Details

JWT Authentication (security.py)

The security module supports two verification modes: (1) JWKS-based RS256 verification using Supabase's public key endpoint (https://[project].supabase.co/auth/v1/.well-known/jwks.json) — the default for modern Supabase projects; (2) HS256 fallback using the SUPABASE_JWT_SECRET shared secret. The get_current_user FastAPI dependency extracts the 'sub' (user UUID) from the verified payload.

AI Service (ai_service.py)

The Groq SDK is initialised once via _get_groq_client() and reused. The prompt builder inserts trip parameters (destination, dates, travellers, budget, currency, travel style, interests, pace, accommodation preference, dietary and accessibility needs, additional instructions) into a structured template. The model is instructed to return only JSON matching the exact itinerary schema. max_tokens=8000, temperature=0.7. Tenacity retries on APITimeoutError, APIConnectionError, and RateLimitError with exponential backoff (3 attempts, 2–30 s wait).

CORS Middleware (main.py)

CORSMiddleware uses allow_origin_regex with an anchored pattern that accepts: http://localhost:, the stable production Vercel URL derived from VERCEL_PROJECT_SLUG, and all Vercel preview URLs matching -.vercel.app. allow_credentials=True; allow_methods=[GET, POST, PATCH, DELETE, OPTIONS]; allow_headers=[Authorization, Content-Type, Accept].

2. Frontend Key Implementation Details

API Client (api.ts)

The ApiClient class wraps all fetch calls. It reads VITE_API_BASE_URL for production (Render URL) or falls back to empty string (Vite proxy in dev). Every request includes Authorization: Bearer obtained from the Supabase session in AuthContext. HTTP errors are caught and converted to typed error objects.

Trip Form Validation (CreateTripPage.tsx)

React Hook Form with a Zod schema validates all fields before submission: destination (non-empty string), start_date and end_date (valid ISO dates, end >= start), travelers (1–50), total_budget (positive number), currency (3-char), travel_style (enum), interests (string array), daily_pace (enum).

3. Database Schema Summary

Column	Type	Constraint
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()

user_id	UUID	NOT NULL, REFERENCES auth.users(id) ON DELETE CASCADE
title	TEXT	NOT NULL
destination	TEXT	NOT NULL
start_date / end_date	DATE	NOT NULL, end_date >= start_date
travelers	INTEGER	1–50
total_budget	NUMERIC(12,2)	NOT NULL, >= 0
travel_style	TEXT	CHECK IN (budget, balanced, luxury, adventure, relaxed, cultural, romantic)
daily_pace	TEXT	CHECK IN (slow, moderate, fast)
days_json	JSONB	DEFAULT '[]' — stores all itinerary days
budget_breakdown	JSONB	Generated by budget_service.py
created_at / updated_at	TIMESTAMPTZ	NOT NULL, auto-updated by trigger